Lab 3: Component Modelling & Architectural Pattern Selection

Remote Patient Vitals Alert & Monitoring App

PES2UG24CS323
NIRAV CHOUDHARY

SECTION F

Scenario: A remote patient uses the app to send vital readings. An on-call caregiver can view the readings and receive alerts when a patient's vitals go outside the safe range.

1. Architectural style analysis

Layered Architecture: This is simple and easy to understand. It separates the app, business logic and data. The downside is that scaling individual parts can be harder.

Microservices Architecture: Different services can handle monitoring, alerts and patient data separately. It can scale well and one service failing does not have to stop everything, but it is more complicated to build and maintain.

Client-Server Architecture: Patient and caregiver apps communicate with a central server. It is simple and keeps patient data in one place, but the server can become a bottleneck or single point of failure.

2. Architecture selection

We chose Client-Server Architecture for the Remote Patient Vitals Alert & Monitoring App.

The main reason is that both the patient and caregiver need to communicate with one central system. The server can receive vital readings, store them and send alerts to the caregiver. It also makes data management easier because patient information is kept in one controlled location.

For security, the server can handle authentication and authorization so only the patient and assigned caregiver can access the data. For performance, the server can process readings and alerts centrally instead of making the mobile apps do all the work.

3. Main components

Remote Patient App - sends patient vital readings to the server.

Monitoring Server - receives readings, checks them and handles the main application logic.

Vitals Database - stores patient details and vital readings.

Alert Service - creates and sends alerts when readings cross a defined limit.

Caregiver App - lets the on-call caregiver view patient readings and receive alerts.

4. Main interfaces

Patient App -> Monitoring Server: Vitals API. Sends heart rate, blood pressure, oxygen level and other readings.

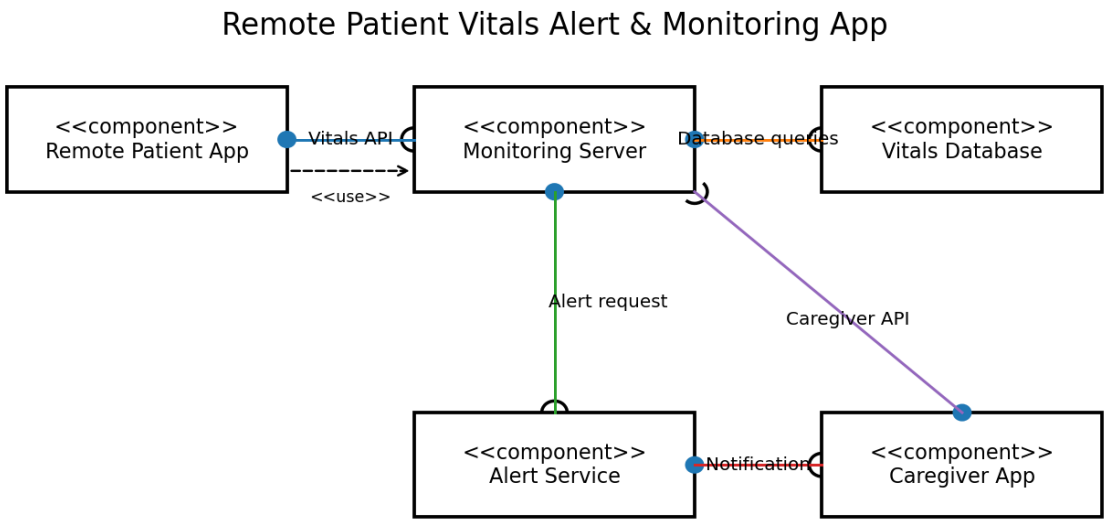
Monitoring Server -> Vitals Database: Database queries. Stores and retrieves patient readings.

Monitoring Server -> Alert Service: Alert request. Sends abnormal readings for alert processing.

Alert Service -> Caregiver App: Notification interface. Sends urgent alerts to the caregiver.

Caregiver App -> Monitoring Server: Caregiver API. Requests patient readings and monitoring information.

5. Component diagram



The diagram shows the main components and how they communicate. The patient app sends readings to the monitoring server. The server stores the data, checks the readings and sends abnormal readings to the alert service. The caregiver app receives alerts and can request the patient's latest information.

This follows the lab requirement to identify components, show interfaces and dependencies, and explain the security and performance effects of the selected architecture.